Commandes en ligne (plateformes de livraison et webshop)

Plateforme : Android 13 (API 33), terminal Sunmi D3 Pro (variante avec services Google) **Date :** À effectuer (EV-18 : date de dépôt) **Code :** `crates/pos-engine/src/commands/webshop.rs`, `crates/pos-engine/src/commands/integrations.rs`, `crates/pos-hubrise/src/webshop.rs` **Références SPF :** FAQ producteurs Q203, Q204 a/b, Q206, Q209, Q211, Q212, Q213, Q301

---

1. Périmètre

Les commandes passées hors de la caisse (plateformes de livraison telles que Deliveroo ou Uber Eats, webshop propre de l'établissement) arrivent dans pili-pos-android par HubRise, un service tiers de concentration de commandes. Le code est celui du moteur commun, identique à celui de `pili-pos` : l'application Android ne change que l'étape de connexion (§11) et l'écran des commandes (§12). Chaque commande en ligne se termine par un événement N signé par le FDM (FAQ Q203 a), créé sur ce terminal, au nom de l'employé qui l'accepte.

L'intégration fait partie du binaire certifié (feature Cargo `cert`, qui inclut `hubrise`). Il n'existe qu'un seul binaire : son fonctionnement se règle par installation (§2), jamais par une autre build.

2. Activation par installation

Élément	Comportement
Valeur de départ	Clé `hubrise_enabled` du `pos.json` du lot de configuration signé, `false` par défaut.
Changement	Un Gérant (rôle `admin`), connecté avec son PIN, dans Réglages > Intégrations. Un Serveur ou un Apprenti ne peut pas.
Priorité	Le dernier changement l'emporte. Une nouvelle valeur de `pos.json`, lue au démarrage du moteur (donc après l'import d'un nouveau lot), compte comme un changement.
Journal	Chaque changement est inscrit dans `integration_switch_log` (date, valeur, source `config` ou `admin`, INSZ du Gérant). Table en ajout seul : des déclencheurs SQLite refusent toute modification ou suppression. Le journal est affiché dans Réglages.
Client HubRise	Section `hubrise` de `pos.json` : identifiant du client OAuth et fichier de son secret, qui voyage dans le lot signé (`MANUEL_INSTALLATION_FR.md` §4.2). Sans elle, la connexion à HubRise est refusée.
Coupure refusée	Tant qu'une commande en ligne n'est pas soldée (en attente, en signature, enregistrée P, remboursement ou clôture à faire), la coupure est refusée avec le nombre de commandes concernées.

Intégration coupée :

- aucun appel vers HubRise (ni réception, ni envoi de statut) ;
- aucune nouvelle commande en ligne acceptée, refusée ou synchronisée ;
- les ventes de la caisse ne sont pas transmises à HubRise ;
- l'écran des commandes en ligne et les réglages HubRise sont masqués dès qu'il ne reste aucune commande en ligne à solder.

Une commande déjà signée reste toujours soldable, intégration coupée : encaissement d'une commande enregistrée (P puis N), remboursement d'une commande annulée, clôture à 0,00 EUR. Aucune vente signée ne peut donc rester sans suite.

Code : `crates/pos-engine/src/commands/integrations.rs` (`require_hubrise`, `do_set_hubrise_enabled`), `crates/pos-db/src/queries/integrations.rs`, `crates/pos-hubrise/src/switch.rs`.

3. Réception et contrôle

1. HubRise transmet la commande ; pili-pos l'enregistre comme commande en attente (table `web-shop_orders`, état `pending`) et répond `received`. Rien n'est signé à ce stade.
2. Chaque article, option et frais est rapproché du menu de la caisse (le menu de la caisse est le reflet du catalogue HubRise, §8). Le taux de TVA vient du menu de la caisse (sur place, ou à emporter pour la livraison et l'enlèvement) et doit correspondre à celui de la commande. La somme des lignes doit égaler le total de la commande.
3. Le mode de paiement est classé :
   - **payée** : les paiements couvrent le total et aucun n'est en espèces ;
   - **à payer** : aucun paiement, ou paiement en espèces à la livraison ou à l'enlèvement.
4. Tout écart (article inconnu, TVA différente, total incohérent, paiement ambigu, canal inconnu) place la commande « à vérifier » : elle ne peut pas être acceptée, seulement refusée ou resynchronisée.

Les commandes en ligne n'apparaissent jamais dans les tables ni dans les ventes à emporter de la caisse, et la caisse ne peut ni les encaisser comme une vente directe ni proposer les types de paiement APP ou ONLINE.

3 bis. Données de clients

Intégration activée, le POS traite des données de clients (voir `SECURITE_BD_FR.md` §6) :

- chaque commande reçue est conservée telle que HubRise l'envoie (`orders.raw_hubrise_json`, `hubrise_events.payload_json`), avec le nom, le téléphone et l'adresse du client ; aucune purge automatique ;
- le ticket client non fiscal (`pos_print::customer_slip`) imprime le nom, le téléphone et l'adresse ; le ticket cuisine (`pos_print::kitchen_ticket`) imprime les lignes et les notes ;
- les ventes de la caisse sont mises en file dans `hubrise_outbox` et envoyées à HubRise (lignes, totaux, paiements, sans données de client) ;
- le jeton d'accès HubRise (`hubrise_connection.access_token`) est stocké en clair dans `pos.db`, donc présent dans les sauvegardes et dans l'export du Gérant pour le SPF (`SECURITE_BD_FR.md` §8). Sur Android, la base est dans le stockage privé de l'application : aucun autre programme ni compte du terminal ne la lit. L'export nocturne n'envoie que la sauvegarde chiffrée et une copie statistique sans donnée de client (`EXPORT_NOCTURNE_FR.md` §5 et §6.3).

4. Événements signés

Situation	Moment	Événement	Paiement	Ticket
Commande payée (plateforme)	Acceptation	N	`APP`	Ticket de caisse TVA papier
Commande payée (webshop)	Acceptation	N	`ONLINE`	Ticket de caisse TVA papier
Commande à payer	Acceptation	P (`signOrder`)	aucun	Bon de cuisine (« ceci n'est pas un ticket de caisse TVA valable »)
Commande à payer	Remise et encaissement	N, qui clôt le P (FAQ Q203 b)	espèces, carte, titres-repas, comme à la caisse	Ticket de caisse TVA papier

Situation	Moment	Événement	Paiement	Ticket
Commande refusée	Refus	aucun	aucun	aucun

Chaque commande porte un `costCenter` :

- `type` : `PLATFORM` pour une plateforme, `WEBSHOP` pour le webshop propre ;
- `id` : le canal (par exemple `DELIVER00`) ;
- `reference` : l'identifiant HubRise de la commande, unique et identique pour le P et le N d'une même commande (FAQ Q211, Q212).

Le `userId` est l'INSZ de l'employé qui accepte (rôle Serveur ou Gérant ; un Apprenti ne peut pas accepter) et le `deviceId` celui du terminal : l'événement est créé sur ce terminal. L'identité système `00000000029` n'ouvre que la fiche de la commande reçue, avant toute acceptation ; elle n'est l'opérateur d'aucun événement signé.

Les lignes sont celles commandées par le client, consolidées (FAQ Q213 b). Les modifications faites par le client sur la plateforme ne sont pas des corrections de l'employé et ne sont pas enregistrées comme telles (FAQ Q213 a) : une modification reçue après la signature place la commande « à vérifier », sans rien signer.

L'acceptation est refusée en mode formation : une commande en ligne est toujours une vente réelle.

5. Remises et frais

- **Remises** (code promo, réduction du webshop ou de la plateforme) : envoyées au FDM comme `priceChanges` de portée `EVENT`, type `PUBLIC`, réparties sur les lignes de produits au prorata de leur montant (méthode du plus grand reste, au cent près). Le ticket affiche la remise, et les totaux par taux de TVA sont nets. Enregistrées dans `order_price_changes` (ajout seul).
- **Frais de livraison ou de service** : un frais facturé par l'établissement est vendu comme une ligne, sa TVA répartie sur les taux des produits de la commande au prorata de leur montant net. Un frais perçu par la plateforme n'est pas le chiffre d'affaires de l'établissement : il est exclu par défaut pour les plateformes (`hubrise_channels.charge_policy`), et un Gérant peut l'inclure canal par canal.

6. Annulations et remboursements

Annulation reçue	Traitement
Avant l'acceptation	La commande est annulée localement. Rien n'est signé.
Après le N (commande payée)	La commande passe « remboursement à faire ». Rien n'est signé automatiquement. Un Gérant confirme le remboursement avec son PIN : un N de remboursement (`negQuantityReason REFUND`) reprend chaque ligne au prix de la vente, frais et remises compris, avec la `fdmRef` de la vente d'origine (FAQ Q301 a). Paiement `APP` ou `ONLINE` : la plateforme ou le prestataire de paiement rembourse le client, le tiroir reste fermé.
Après le P (commande à payer)	La commande passe « clôture à faire ». Chaque ligne est corrigée (`CORRECTION`), puis un N à 0,00 EUR avec un tableau `financials` vide (FAQ Q206 b) clôt le P avec le même `costCenter`, et son ticket de caisse TVA est imprimé (FAQ Q206, Q209).

Une annulation reçue pendant que la caisse était éteinte est trouvée au démarrage suivant : pili-pos-android relit les commandes signées des deux derniers jours.

7. Pannes

- **FDM injoignable ou refus du FDM** à l'acceptation : rien n'est signé, la commande revient en attente, HubRise n'est pas informé, et le même message d'erreur qu'à la caisse est affiché. L'acceptation peut être relancée.
- **Double appui ou événement reçu deux fois** : la commande est réservée (`pending` vers `signing`) avant l'appel au FDM ; une seconde acceptation est refusée.
- **Arrêt brutal pendant la signature** : au démarrage, une commande restée en `signing` est marquée signée si l'archive fiscale contient son événement, sinon elle revient en attente. Un remboursement ou une clôture interrompus sont terminés ou proposés à nouveau.
- **HubRise injoignable après la signature** : la vente reste signée ; le statut à envoyer attend dans une file locale et part au retour de HubRise.

8. Menu de la caisse

Quand HubRise est connecté, le menu de la caisse est le reflet du catalogue HubRise : chaque article garde son identifiant (le `productId` envoyé au FDM) d'une synchronisation à l'autre, un article retiré est désactivé et jamais supprimé, et un catalogue invalide (taux de TVA hors A/B/C/D, référence manquante, prix invalide) est refusé en entier, le menu précédent restant en place. La modification du menu sur la caisse est bloquée pendant la connexion.

Les taux de TVA : 21 % donne A, 12 % donne B, 6 % donne C, 0 % donne D. Le taux sur place fixe la TVA d'une consommation sur place, le taux à emporter celle de la livraison et de l'enlèvement.

9. Schéma

Migrations 20 à 25 (`crates/pos-db/src/schema_hubrise.rs`, `crates/pos-db/src/schema_catalog.rs`), détaillées dans `SCHEMA_BD_FR.md` §5 :

Version	Contenu
20	File d'envoi vers HubRise (ordre par commande, nouvelles tentatives), état de la connexion
21	Menu reflet d'un catalogue externe (articles actifs, catégories masquées, état de synchronisation)
22	`webshop_orders`, `hubrise_channels`, `hubrise_cash_payment_refs`
23	`order_price_changes` (ajout seul)
24	Frais des plateformes exclus par défaut
25	`integration_switches`, `integration_switch_log` (ajout seul)

10. Preuves

Tests d'intégration, exécutés contre un FDM simulé et un HubRise simulé dans le processus (`cargo test -p pos-engine --no-default-features --features cert`) :

Fichier	Couvre
`crates/pos-engine/tests/hubrise_orders_ingest.rs`	Réception, contrôle, « à vérifier », exclusion des tables et ventes à emporter
`crates/pos-engine/tests/hubrise_orders_signing.rs`	N à l'acceptation (APP, ONLINE), TVA B sur place et C en livraison, P puis N à la remise avec le même `costCenter`, refus jamais signé, double appui, FDM injoignable, arrêt brutal, mode formation et Apprenti refusés

Fichier	Couvre
`crates/pos-engine/tests/hubrise_cancellations.rs`	Annulation avant acceptation, remboursement par un Gérant avec `fdmRef`, clôture d'un P par corrections et N à 0,00 EUR, annulation trouvée au démarrage
`crates/pos-engine/tests/hubrise_discounts.rs`	Remises en `priceChanges` sur le N, le P et le remboursement
`crates/pos-engine/tests/hubrise_switch.rs`	Intégration coupée : aucun appel, aucune acceptation ; une commande signée reste soldable ; changements par un Gérant seulement, journalisés ; coupure refusée tant qu'une commande n'est pas soldée
`crates/pos-engine/tests/hubrise_catalog.rs`	Menu reflet du catalogue, identifiants stables, catalogue invalide refusé

Les mêmes parcours (catalogue, commande payée annulée puis remboursée, commande à payer encaissée à la remise) ont été rejoués contre l'API HubRise réelle, sur un établissement de test, avec le moteur commun ; les réponses enregistrées servent à vérifier que le HubRise simulé a la même forme (`crates/pos-hubrise-mock/tests/hubrise_shape.rs`). Aucun appel réel à HubRise n'est fait depuis l'application Android pour ce dossier. Les tests de l'écran sont dans `android/app/src/test/kotlin/dev/nuagemagique/pilipos/feature/` (`settings`, `webshop`). Captures des deux écrans sur le D3 Pro émulé : A52 et A112 à A114 de *CAPTURES_ECRAN_FR.pdf*.

11. Connexion à HubRise sur Android

Écran Réglages (`docs/android/parity/settings.md`), panneau « Intégrations » puis section HubRise. Le terminal n'a pas de navigateur et n'ouvre aucun lien :

1. L'intégration étant activée par un Gérant (§2), l'utilisateur touche « Se connecter à HubRise ».
2. Le moteur construit le lien d'autorisation (`hubrise_start_oauth_flow`) et l'application le dessine en code QR (`qrMatrix`, `FFI.md` §9).
3. Il scanne le code avec un téléphone, se connecte à HubRise et choisit l'établissement ; HubRise affiche un code d'autorisation.
4. Il tape ce code dans le champ « Code d'autorisation » ; un code vide est refusé. Le moteur l'échange contre le jeton (`hubrise_complete_oauth`, HTTPS avec les racines embarquées de l'APK : `EXPORT_NOCTURNE_FR.md` §7).
5. À la première connexion, un Gérant choisit les produits qui gardent leur identifiant, puis les canaux (§5, §8). Se déconnecter demande une confirmation.

12. Écran des commandes en ligne sur Android

Le bouton « En ligne » du bandeau, avec le nombre de commandes en attente, ouvre l'écran (`docs/android/parity/webshop`). Il liste les commandes du moteur (`hubrise_list_webshop_orders`) et suit ses événements ; il ne calcule aucun montant. Colonnes « Nouvelles », « En cuisine », « Prêtes » (et « Terminées » sur demande). Chaque carte montre le canal en texte, le service, l'âge et les actions du moteur : « Accepter » (§4), « Préparer », « Prête », « En livraison », « Terminer », « Refuser », « Retarder » (+15, +30, +45 min), « Ticket cuisine », « Ticket client ». « Refuser » et « Rembourser » (§6, Gérant seulement) demandent une confirmation. Le carillon d'arrivée d'une commande (`OrderChime`) est joué par l'application.